

top-left to the bottom-right, are identical. For example, consider the matrix:

$$\begin{bmatrix} a & b & c & d \\ e & a & b & c \\ f & e & a & b \\ g & f & e & a \end{bmatrix}$$

In this matrix, every diagonal is filled with a consistent value -- the first diagonal holds the value *a*, the second holds *b*, and so on. This property is known as diagonal uniformity. A notable aspect of a Toeplitz matrix is its straightforwardness and consistency—knowing the first row and the first column allows for the complete determination of the entire matrix. This repetitive and organised structure forms the basis for the investigation of Toeplitz networks in graph theory, where similar patterns of repetition and uniformity are explored.

Toeplitz networks possess several important graph-theoretical characteristics that make them valuable for both theoretical exploration and practical implementation. A primary characteristic is domination, which allows for the selection of a subset of vertices such that every vertex in the network is either included in this subset or is adjacent to a vertex within it. The inherent structured repetition found in Toeplitz networks facilitates the efficient analysis and reduction of dominating sets. Additionally, connectivity is notably predictable due to the network's uniform structure, which aids in examining paths, flows, and the overall resilience of the network. In certain instances, Toeplitz networks may also demonstrate planarity, enabling their representation on a plane without edge crossings, thereby enhancing visualisation and streamlining specific computational processes. These properties, combined with the symmetry and regularity of the network, make Toeplitz networks ideal for modelling and optimising systems with periodic or repeating interactions. Understanding these networks not only deepens mathematical insight but also supports practical advancements in designing robust and efficient systems.

Introducing SageMath

SageMath is an extensive, open source mathematical software platform that offers robust capabilities for conducting a diverse range of computations in fields such as algebra, calculus, number theory, and graph theory. With a focus on accessibility and flexibility, SageMath combines various mathematical libraries and tools, including NumPy, SciPy, Matplotlib, and dedicated graph theory packages, to create a cohesive environment for both symbolic and

numerical analysis. Its interface, built on Python, facilitates user engagement by enabling script writing, algorithm development, and the automation of intricate tasks.

SageMath facilitates the straightforward creation, analysis, and visualisation of graphs. When examining structures such as Toeplitz networks, it offers tools for constructing graphs with designated patterns, calculating properties like domination numbers and connectivity, and visualising these networks to uncover structural insights. Its capacity for managing large-scale computations positions it as an excellent option for addressing intricate problems in both theoretical research and practical applications. Furthermore, the open source nature of SageMath promotes customisation and collaboration, making it an invaluable asset for educators, researchers, and students alike. By effectively linking theoretical enquiry with computational prowess, SageMath enables users to investigate and innovate across a diverse array of mathematical challenges.

Modelling a Toeplitz network using SageMath

SageMath's flexibility and user-friendly interface position it as the perfect resource for exploring Toeplitz networks. By enabling the integration of graph creation, property evaluation, and visualisation within one platform, it minimises the need to transition between different tools, thereby boosting efficiency. Furthermore, its Python-based syntax makes it approachable for those with programming experience, and its open source nature fosters collaboration and customisation to meet specific research requirements. You can use the following code to model a Toeplitz network using SageMath.

```
# Importing the graph module
from sage.graphs.graph import Graph

# Define the number of vertices
n = 10

# Create an empty graph
G = Graph()

# Add vertices and edges with periodic connections
for i in range(n):
    G.add_vertex(i)
    G.add_edge(i, (i + 1) % n) # Connect to the next vertex (wrap-around)
    G.add_edge(i, (i + 2) % n) # Connect to the second next vertex (wrap-around)

# Visualize the graph
G.show()
```